

TRABAJO PRÁCTICO INTEGRADOR: VIRTUALIZACIÓN

Alumnos: Theo Wlasiczuk, Facundo Gonzalez

Materia: Arquitectura y sistemas operativos

Profesor: Carlos Andrés Odiard

Fecha de entrega: 25/06/2026

ÍNDICE

1. INTRODUCCION.....	3
2. MARCO TEORICO.....	3
3. CASO PRACTICO.....	4
4. METODOLOGIA UTILIZADA.....	8
5. RESULTADOS OBTENIDOS.....	9
6. CONCLUSIONES.....	9
7. BIBLIOGRAFIA.....	10

1. INTRODUCCION

Se eligió el tema de virtualización ya que ya había conocimiento previo sobre su uso y desarrollo. En cuanto a la importancia que tiene y cómo influye en la formación hacia técnico en programación, nos ayudó a poner en práctica nuestras habilidades a la hora de gestionar conexiones remotas a servidores, manejo de máquinas virtuales y administración de trabajo colaborativo.

A modo de objetivo, nos propusimos facilitar el trabajo colaborativo en servidores, haciendo uso de herramientas como SSH y redes privadas virtuales para permitir el acceso remoto.

2. MARCO TEORICO

VIRTUALIZACIÓN

La virtualización es una herramienta que permite ejecutar sistemas operativos dentro de otro que cumple la función de "host". Este utiliza un "hipervisor": una capa de software encargada de asignar recursos del sistema base al o los sistemas virtualizados.

Existen dos tipos de hipervisores, de tipo 1 que corren directamente en el hardware, que se utilizan generalmente en servidores que ofrecen IaaS (por ejemplo una VPS). Y los de tipo 2 que podrían ser, por ejemplo, VirtualBox que se ejecutan como una aplicación dentro de un sistema operativo ya instalado.

Estos hipervisores de tipo 2 permiten, entre muchas otras cosas, configurar de manera cómoda, por ejemplo, la configuración de red. Por ejemplo, VirtualBox muestra tres alternativas relacionadas a esto:

- **Modo Bridge:** Permite elegir una interfaz de red que ya tenga el host, para que la VM actúe como un dispositivo más de esa red.
- **Modo NAT:** Aísla la VM de la red local, redirigiendo su conexión al gateway del sistema host, permitiendo que solo acceda a internet.
- **Modo Host-only:** Una red que comunica únicamente la máquina virtual con el host.

ZEROTIER

ZeroTier es una herramienta que permite la creación de redes privadas virtuales facilitando la conexión remota entre dispositivos, que "simula" una red local entre los dispositivos que estén conectados a ella. Y a diferencia de una VPN común que trabaja con un servidor central, ZeroTier prefiere (en los casos que lo permitan) la conexión directa entre los dispositivos, ofreciendo menos latencia en la conexión y reduciendo la necesidad de realizar configuraciones de puertos o firewall.

NGINX

Utilidad que permite la creación de servidores web, proxy inverso o incluso como balanceador de carga. Se suele preferir sobre, por ejemplo Apache, debido al bajo consumo de recursos y el manejo de gran cantidad de conexiones simultáneas.

SSH

Secure Shell es un protocolo que permite una conexión remota segura entre dos dispositivos, que permite el acceso a una terminal del dispositivo remoto como si se estuviera usando directamente. Es ampliamente utilizada en la administración de servidores debido a la seguridad que este ofrece, ya que cifra toda la conexión, y permite la utilización de claves públicas/privadas en lugar de contraseñas, para mejorar aún más la seguridad.

3. CASO PRACTICO

Como caso práctico del trabajo, se planteó habilitar el acceso colaborativo a la máquina virtual creada desde dos ubicaciones físicas distintas, para que ambos integrantes podamos trabajar con este de manera simultánea. Para esto se empleó la utilización de SSH como protocolo de acceso remoto, y ZeroTier como proveedor de red privada para permitir la conexión remota.

Como plataforma de hipervisor se eligió Oracle VirtualBox, corriendo en un host Linux, donde se configuró una máquina virtual de Ubuntu Server 22.04 con 2 núcleos y 2GB de RAM asignados.

Un paso importante a mencionar es que, al momento de realizar la instalación del sistema operativo, en uno de los pasos se utilizó GitHub como proveedor de claves públicas, ya que el instalador permite importarlas desde la plataforma usando el nombre de usuario. Esto funciona siempre y cuando la cuenta de GitHub tenga cargada claves públicas en su configuración.

Una vez instalado el sistema operativo, se procedió con la instalación de los paquetes necesarios que no vienen por default, como son *nginx*, *python* y *zerotier*, utilizando su respectivo comando:

sudo apt install nginx python

Para ZeroTier, en cambio, como no se encuentra en el repositorio oficial de Ubuntu se utilizó el script de instalación especificado en la documentación del programa:

curl -s https://install.zerotier.com | sudo bash

Ya teniendo todo instalado toca configurar tanto ZeroTier como el servidor Nginx. Para el primero se necesita antes que nada ingresar al panel de control web necesario para la

creación de la red, siguiendo estos pasos:

1. Entrar a <https://my.zerotier.com/network/>
2. Presionar el botón "Create A Network", que nos dará una network id (ej: e4da7455b29ab169)
3. En en servidor y la pc remota ejecutar el siguiente comando:
`zerotier-cli join e4da7455b29ab169`

Nota: en Windows existe un acceso al programa a través de la barra de tareas con el que se puede realizar la misma operación.

Una vez logrados estos pasos, se encontrará habilitada la posibilidad de conectarse a la máquina virtual de forma remota.

El siguiente gráfico representa de manera general cómo están conectadas cada PC al servidor:

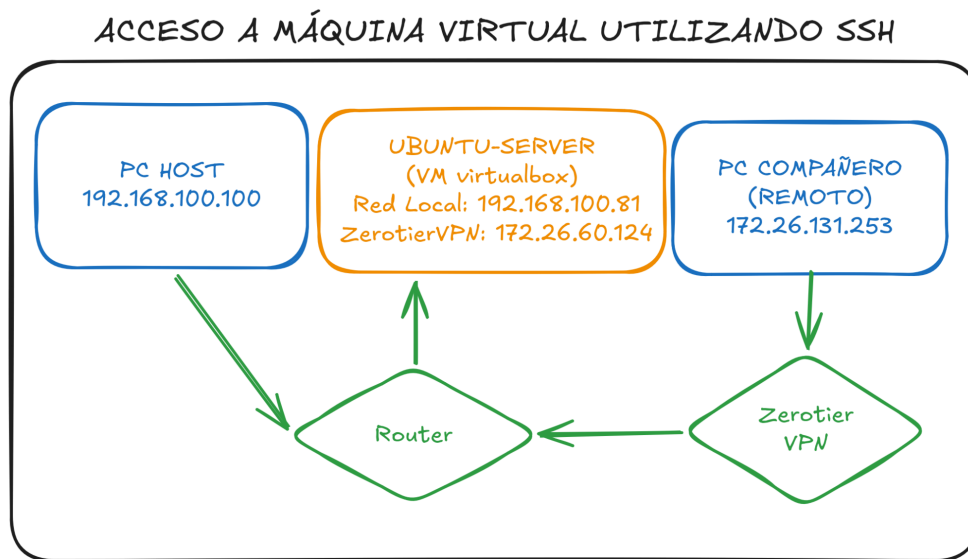


Figura 1: Diagrama de conexión de los dispositivos

Para la configuración de red, se optó por configurar la máquina virtual en modo *bridge* para poder acceder tanto de forma local como remota.

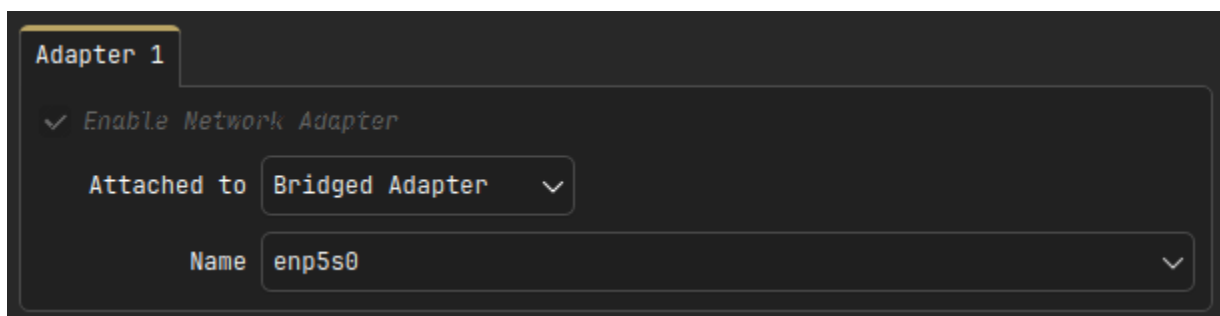


Figura 2: configuración de red de la VM, seteada en modo bridge en la red local del host

```
2: enp0s3: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
    link/ether 08:00:27:a9:6b:b5 brd ff:ff:ff:ff:ff:ff
    inet 192.168.100.81/24 metric 100 brd 192.168.100.255 scope global dynamic enp0s3
        valid_lft 84102sec preferred_lft 84102sec
    inet6 fe80::a00:27ff:fea9:6bb5/64 scope link
        valid_lft forever preferred_lft forever
3: ztwfuo5zg3: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 2800 qdisc fq_codel state UNKNOWN group default qlen 1000
    link/ether 6a:c7:1b:f7:ed:d2 brd ff:ff:ff:ff:ff:ff
    inet 172.26.60.124/16 brd 172.26.255.255 scope global ztwfuo5zg3
        valid_lft forever preferred_lft forever
    inet6 fe80::68c7:1bff:fef7:edd2/64 scope link
        valid_lft forever preferred_lft forever
```

Figura 3: conexiones de red de la VM

- *enp9s3*: Conexión de la red local
- *ztwfuo5zg3*: Conexión de la red privada de ZeroTier

Así también se decidió instalar NGINX, para utilizarlo como servidor HTTP básico, para exponer una página estática.

A modo de verificación temprana se probó acceder al servidor NGINX tanto desde la pc host (linux) como la remota (windows):

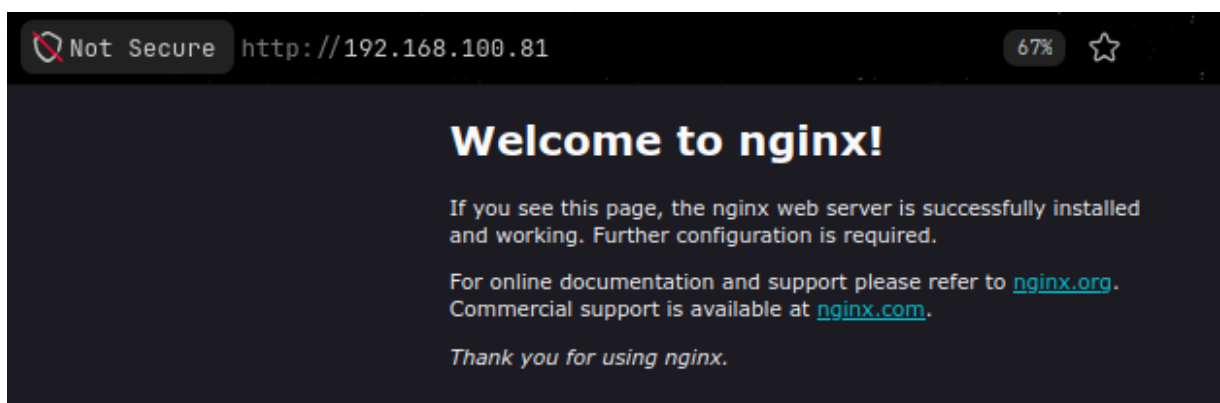


Figura 4: acceso al servidor HTTP desde el PC host

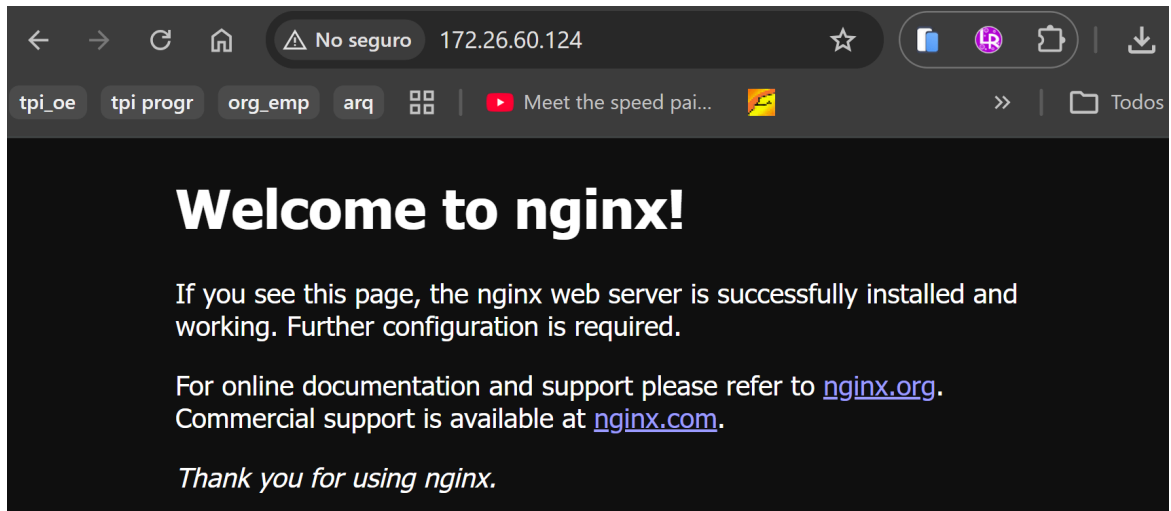


Figura 5: acceso al servidor HTTP desde la PC remota

Página estática personalizada en nginx:

1. ***scp "C:\Users\lucas\Downloads\homepage.html" ubuser@172.26.60.124:/home/ubuser***
Subimos el archivo HTML a la carpeta home del servidor.
2. ***sudo mv homepage.html /var/www/html***
Movemos el archivo hacia el directorio `/var/www/html`
3. ***sudo rm index.nginx-debian.html***
Eliminamos el html de inicio de nginx
4. ***sudo mv homepage.html index.nginx-debian.html***
Cambiamos su nombre por el nombre de la página de inicio de nginx que borramos
5. ***sudo systemctl restart nginx.service***
Reiniciamos el servicio nginx

Script de Python:

1. ***scp "C:\Users\lucas\Downloads\main.py" ubuser@172.26.60.124:/home/ubuser***
Subimos el archivo main.py hacia la carpeta home del servidor.
2. ***sudo mkdir scripts***
Creamos la carpeta `scripts` para guardarlo
3. ***mv main.py scripts/***

Movemos el archivo a scripts

4. *cd scripts*

Nos movemos dentro del directorio scripts

5. *python3 main.py*

Corremos el programa

4. METODOLOGIA UTILIZADA

Dado que ambos participantes cuentan con conocimientos previos en máquinas virtuales, la investigación se centró en el uso de ZeroTier para permitir la conexión remota entre PCs, y la configuración de red de la VM para permitir este acceso tanto desde la red local como la privada.

Etapas del desarrollo:

1. Instalación y configuración de VM (Ubuntu Server 22.04) en VirtualBox.
2. Configuración de red en modo bridge, para el acceso local a la VM.
3. Instalación y configuración de ZeroTier en el servidor y la PC remota.
4. Instalación de nginx y publicación de página estática como servicio de prueba.
5. Validación de acceso al servidor desde el ámbito local y remoto.

Con respecto a las herramientas utilizadas fueron las siguientes:

- **Nginx:** utilizado como servidor web.
- **Ssh:** para permitir el trabajo remoto sobre la VM, incluyendo transferencia de archivos y administración por consola.
- **Github:** usado como facilitador de manejo de key públicas para el acceso remoto.
- **ZeroTier:** ZeroTier ayudó a conectar la PCs remota al servidor y permitir que ambos integrantes trabajen en la VM.
- **Oracle VirtualBox:** usado como hipervisor para crear y usar la VM del trabajo.

Reparto de tareas:

Respecto al trabajo colaborativo, la configuración inicial del entorno de trabajo (instalación de vm y configuración de red) fueron realizadas por uno de los integrantes. La configuración de ZeroTier se realizó de manera conjunta, así como el testeo de la conexión SSH, entre la VM y la pc remota, mientras que la personalización del html estático de nginx y, la creación y prueba del script de Python estuvieron a cargo del otro integrante, simulando así una especie de colaboración entre un *sysadmin* y un *desarrollador web*.

5. RESULTADOS OBTENIDOS



Figura 6: Pagina de inicio de Nginx modificada.

```
ubuser@ubuntu-server: ~/scripts
GNU nano 7.2
def par_impar(n):
    if n.isdigit():
        n = int(n)
    else:
        print("Error: Debe ingresar un numero entero.")
        return
    if n == 0:
        print(f"El numero {n} es nulo.")
        return
    if n % 2 == 0:
        print(f"El numero {n} es par.")
    else:
        print(f"El numero {n} es impar.")
par_impar(input("Ingrese un numero entero: "))
```

Figura 7: Visualización con nano de main.py conteniendo su código.

```
ubuser@ubuntu-server:~/scripts$ python3 main.py
Ingrese un numero entero: 5
El numero 5 es impar.
```

Figura 8: El programa en funcionamiento.

6. CONCLUSIONES

A través de este trabajo logramos poner en práctica conceptos fundamentales de la virtualización y de redes, tales como la configuración de máquinas virtuales, el uso de SSH para acceso remoto y la creación de redes privadas con ZeroTier que habilitó esto último.

Esto trabajo nos permitió desarrollar nuestras habilidades en administración de servidores y la colaboración entre integrantes de roles distintos, que nos será útil para seguir con la formación como técnicos en programación.

7. BIBLIOGRAFIA

ArchWiki. VirtualBox. Arch Linux. <https://wiki.archlinux.org/title/VirtualBox>

ZeroTier, Inc. Documentacion de ZeroTier . <https://docs.zerotier.com/>

Canonical Ltd. Documentacion de Ubuntu Server . Ubuntu.

<https://ubuntu.com/server/docs/>

8. LINKS

Repositorio de GitHub:

<https://github.com/FaKsho/ayso-tpi>

Carpeta de Drive con Video de Defensa:

<https://drive.google.com/drive/folders/1fd92jCeNisYsedRIxWh9YcuWreHAXiMk>